

Project 3: Active Learning for Learning-to-Defer

Human-Centric Artificial Intelligence

AG News topic classification with a human-AI team

Generated from the experiment artifacts on 2026-09-10

Experimental setup

All experiments use the AG News dataset (fancyzhx/ag_news): 120,000 training and 7,600 test news articles, evenly distributed over four topics (World, Sports, Business, Sci/Tech). The official training set is split once, stratified with a fixed seed, into 110,000 training and 10,000 validation articles. Every design decision that needs tuning - the deferral threshold in Task 3 and all monitoring during active learning in Task 4 - uses only the validation split. The official test set is reserved for final evaluation, so each reported test number is computed exactly once.

The code is organised as an offline pipeline (`python -m project3.experiments.run_all`) that writes metrics, figures and this report to disk; the project web page displays those artifacts, offers this report for download, and adds an interactive mode where a person can play the expert (Task 5).

Task 1: Baseline classifier

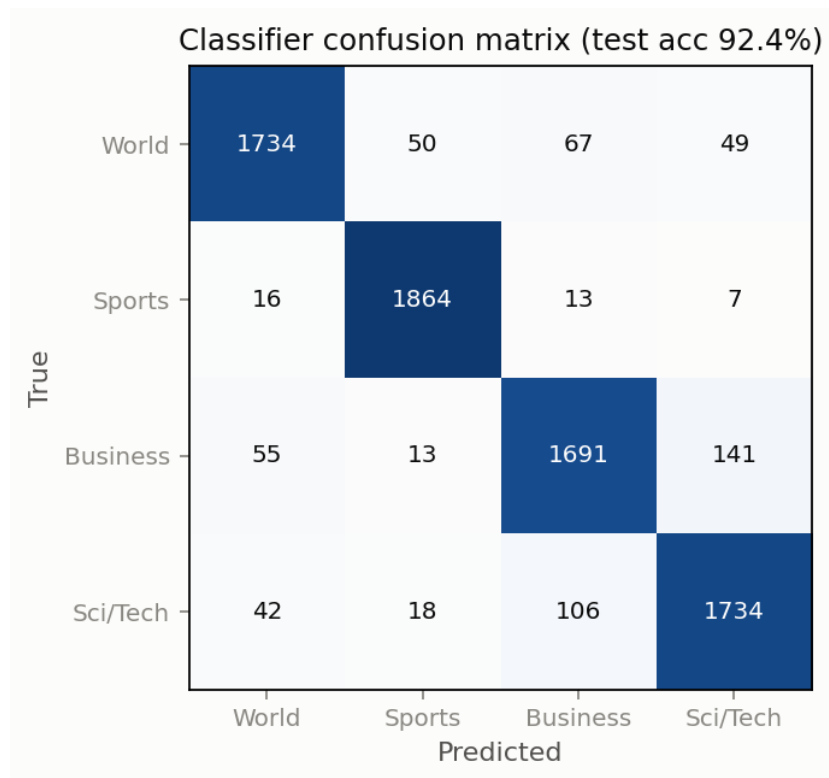
Design

The baseline is a linear classifier over sparse TF-IDF features: word 1-2 grams, sublinear term frequency, 300k features, followed by multinomial logistic regression ($C=4$). Two considerations drove this choice over a fine-tuned transformer. First, practicality: the model trains on 110k documents in about half a minute on CPU, which keeps every later experiment (rejector training, nine active-learning sweeps, the interactive mode) cheap and reproducible. Second, the deferral system needs per-example confidence scores: logistic regression's probability estimates are reasonably calibrated out of the box, whereas a small accuracy gain from a deep model would not change the human-AI comparisons studied here.

Results

The classifier reaches 92.41% accuracy on the test set (validation: 92.77%). Performance is not uniform: Sports is nearly solved, while Business and Sci/Tech are confused with each other most often - exactly the region where a complementary expert can help.

Class	Precision	Recall	F1	Support
World	0.939	0.913	0.926	1900
Sports	0.958	0.981	0.970	1900
Business	0.901	0.890	0.895	1900
Sci/Tech	0.898	0.913	0.905	1900



Task 2: Simulated experts

Design

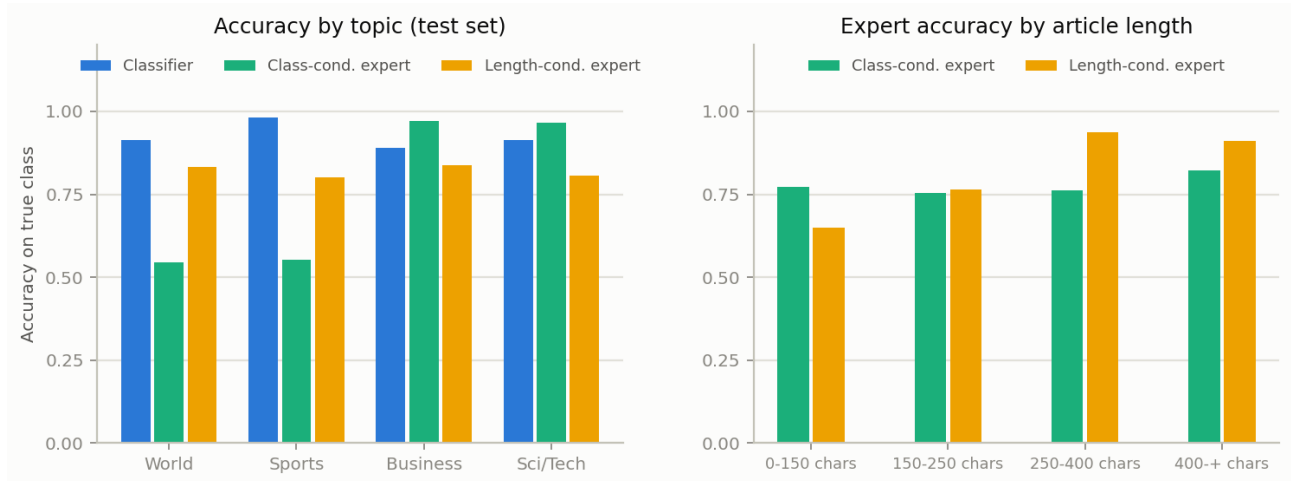
Both experts are imperfect by construction and competent only in a region of the input space. They are deterministic: each answer is derived from a hash of the article's row key, so querying the same article twice always returns the same answer - important because Tasks 3, 4 and 5 must see one consistent expert. When an expert errs, it prefers a plausible confusion (e.g. Business vs Sci/Tech) rather than a uniformly random topic.

The primary expert is class-conditional: Class-conditional expert: 97% correct on [Business, Sci/Tech], 55% on the other topics. It models a business/technology specialist and is deliberately strong exactly where the Task 1 classifier is weakest (Business, Sci/Tech), giving the human-AI team real complementarity. The second expert is length-conditional: Length-conditional expert: 93% correct on articles ≥ 215 chars, 62% on shorter ones. It models a reader who judges long articles reliably but guesses on short snippets, and shows that competence regions need not align with class boundaries.

Results and analysis

On the test set the class-conditional expert reaches 75.9% overall - far below the classifier's 92.4% - yet on its strong classes it is clearly better: 97.2% on Business and 96.6% on Sci/Tech versus the classifier's 89.0% and 91.3%. Its weakness is everything else (54.5% on World, 55.3% on Sports). The length-conditional expert reaches 81.9% overall; its accuracy rises with article length, and neither expert alone can replace the classifier. A deferral policy that knows these profiles, however, can combine the best of both team members.

Expert	Overall	World	Sports	Business	Sci/Tech
Class-conditional	75.9%	54.5%	55.3%	97.2%	96.6%
Length-conditional	81.9%	83.3%	80.1%	83.6%	80.7%



Task 3: Learning to defer

Method

With expert labels available for the whole training set, the system learns when deferral is beneficial with a staged (post-hoc rejector) approach. A binary rejector $g(x)$ is trained on the same TF-IDF features to predict the probability that the expert answers x correctly, using the observed expert answers on the training set. The team then defers exactly when the expert looks more reliable than the classifier: defer iff $g(x) - \max_y p(y|x) > \tau$. The threshold τ is chosen on the validation split (selected value: $\tau = 0.04$).

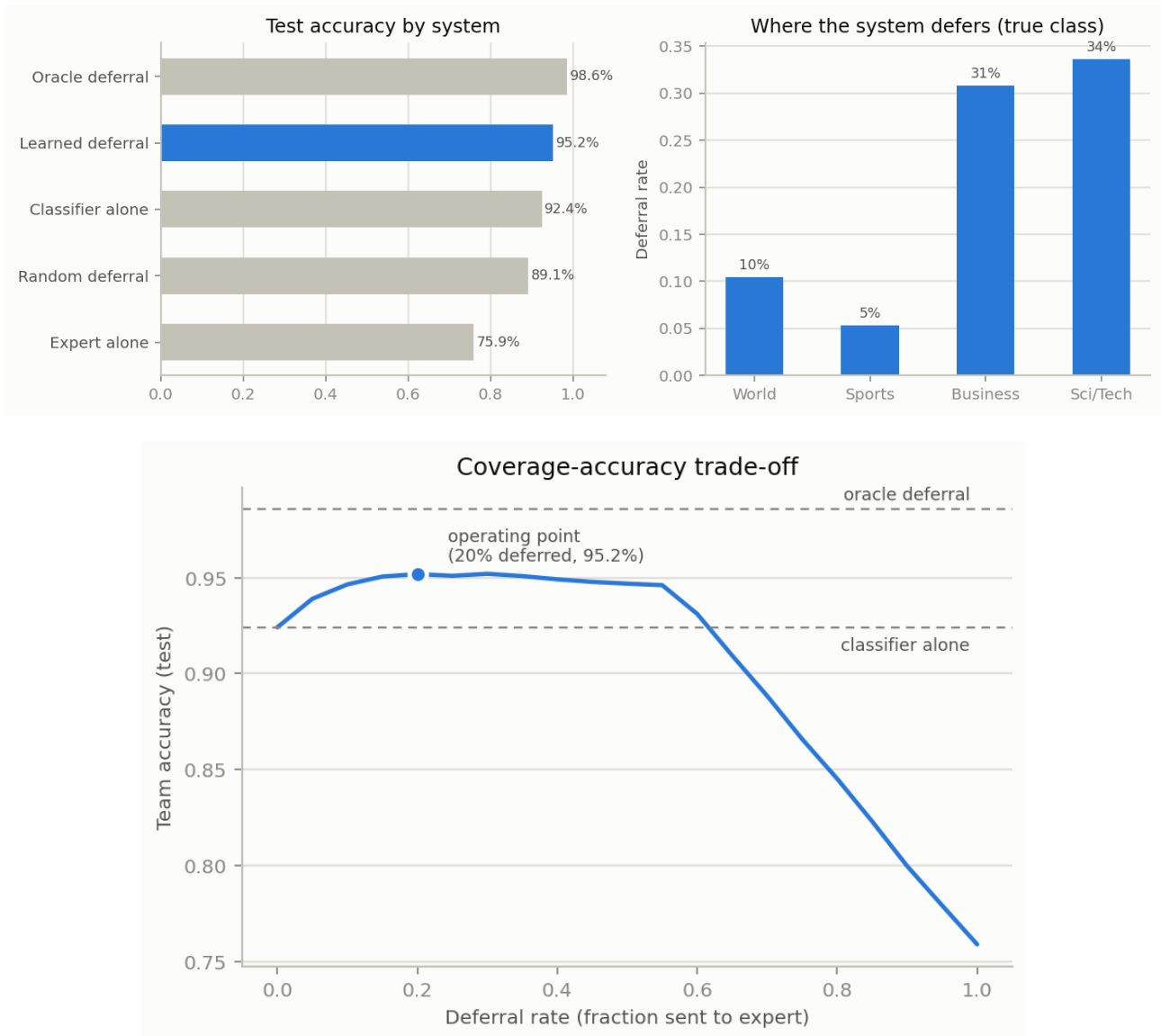
A joint surrogate loss in the style of Mozannar & Sontag (2020) - a $(K+1)$ -way classifier with a defer option trained with a specially weighted loss - is the main alternative. It cannot be expressed with scikit-learn's estimators (no custom per-class loss weighting), and the staged approach has a clear advantage for analysis: $g(x)$ is an explicit, inspectable model of the expert's competence, which Task 4 then learns actively.

Results

The team reaches 95.17% test accuracy at 79.9% coverage: 1,524 of the 7,600 test articles (20.1%) are sent to the expert and the classifier answers the rest. That beats the classifier alone (92.41%), the expert alone (75.88%), and random deferral at the same rate (89.14%); the oracle that defers exactly when it helps sits at 98.57%.

The deferral decisions themselves are sensible on both axes. Quality of the work split: kept articles are classified at 96.99% and deferred ones answered by the expert at 87.93% - both far above what the other team member would achieve on that share. Competence discovery: the system defers 31% of Business and 34% of Sci/Tech articles but only 5% of Sports - it has located the expert's strong region without being told. Judged against the oracle should-defer set (classifier wrong and expert right), deferral recall is 68.2% and precision 20.9%; precision is naturally low at this operating point because deferring an article the classifier would also get right costs the team nothing when the expert is strong there.

System	Test accuracy
Expert alone	75.88%
Random deferral (same rate)	89.14%
Classifier alone	92.41%
Learned deferral (team)	95.17%
Oracle deferral (upper bound)	98.57%



The coverage-accuracy curve (deferring the top- q fraction by rejector score) shows the whole trade-off: team accuracy peaks around 20-30% deferral and degrades gracefully toward the expert-alone extreme, so an operator can pick any workload split for the human expert without retraining.

Task 4: Active learning for expert competence discovery

Setting and strategies

Now no expert labels exist up front. The classifier stays fixed (it already has all gold labels); what is missing is the expert's competence profile, so only the rejector must be learned, from answers obtained by querying the expert. Queries are selected in batches of 100 from a pool of 30,000 training articles; after each batch the rejector is refit on all answers so far and the team is evaluated on the validation split with the fixed rule defer iff $g(x) > \max_y p(y|x)$. Budgets from 25 to 5,000 queries are checkpointed, with 3 seeds per strategy.

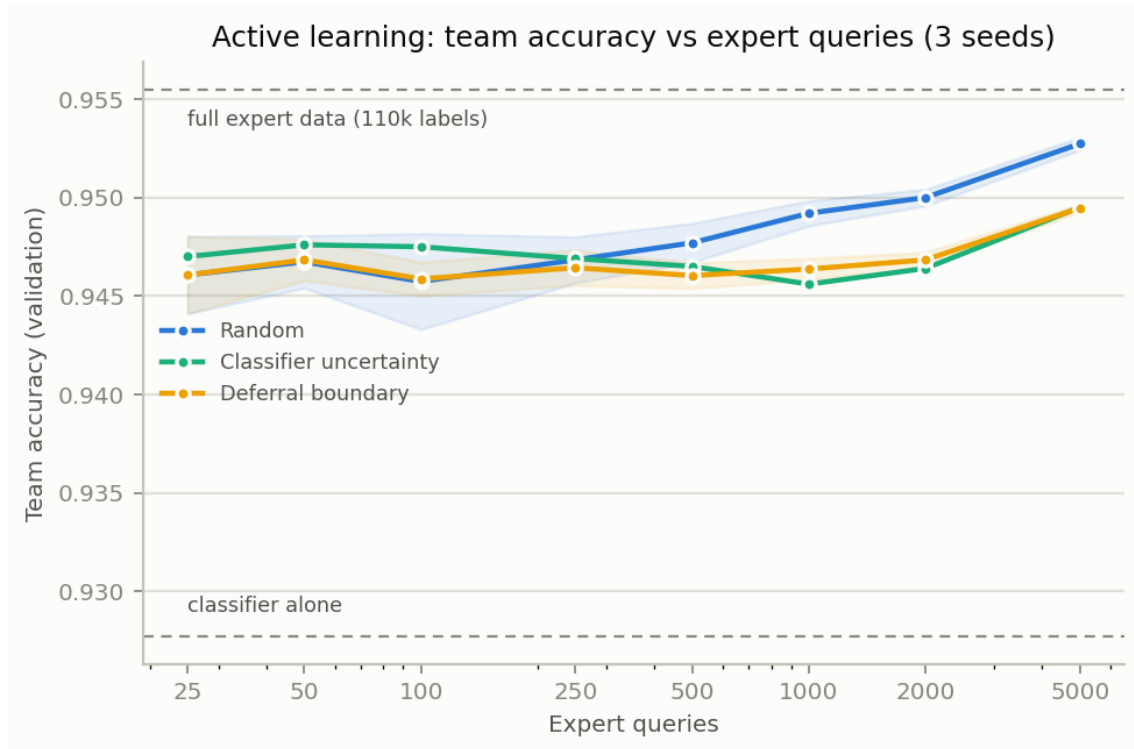
Three query strategies are compared. Random sampling is the baseline. Classifier-uncertainty sampling queries where the classifier is least confident - the classic active-learning heuristic, included because deferral candidates live exactly in that region. The proposed strategy, deferral-boundary sampling, queries where $|g(x) - \max_y p(y|x)|$ is smallest: the expert's answer there is precisely the information that flips a defer/keep decision, which is the quantity we care about (with a first random batch and 10% random exploration per batch).

Results

The headline result is how cheap competence discovery is: with only 25 expert queries every strategy already lifts the team to about 94.6% on validation, versus 92.77% for the classifier alone and 95.55% for a rejector trained on all 110,000 expert labels. Roughly 100 queries recover about 65% of the value of full expert supervision at 0.1% of the labeling cost.

Between strategies the honest finding is that random sampling is the strongest at large budgets (95.27% at 5,000 queries vs 94.95% for deferral-boundary), and no strategy separates from the others at small budgets. The explanation is instructive. This expert's competence is class-determined, and class is linearly recoverable from TF-IDF features, so a handful of random examples per topic already pins the profile down - there is little room for cleverness. Both targeted strategies also collect a deliberately biased sample (hard or boundary articles), which miscalibrates the rejector's probabilities slightly; the deferral rule compares $g(x)$ against classifier confidence directly, so calibration matters more than ranking here. Targeted querying should pay off when competence varies within classes in subtler ways; for this problem the practical recommendation is the simple one - stratified random querying with a few hundred labels.

Strategy	25	50	100	250	500	1000	2000	5000
Random	0.9461	0.9467	0.9457	0.9468	0.9477	0.9492	0.9500	0.9527
Clf. uncertainty	0.9470	0.9476	0.9475	0.9469	0.9465	0.9456	0.9464	0.9495
Deferral boundary	0.9461	0.9468	0.9459	0.9464	0.9460	0.9464	0.9468	0.9495



Task 5: Interactive mode (optional)

The project interface includes a 'Be the expert' mode implementing the optional Task 5. The Task 4 loop runs with a person instead of the simulated expert: the system picks the next article (random warm-up, then deferral-boundary sampling), the user assigns one of the four topics, and the rejector - now a model of that user's competence - is refit after every answer. A live panel shows the user's measured accuracy per topic and the estimated team accuracy if the current rejector were deployed with them as the expert. Because the queried articles come from the labeled training set, the user's answers can be scored against gold labels immediately.

Limitations

The experts are simulations with conveniently simple competence structure; as Task 4 showed, that structure makes competence discovery easy and flatters random querying. Real expert competence drifts, depends on effort and time, and is not a deterministic function of the article. The rejector reuses the classifier's TF-IDF representation, so competence patterns invisible in bag-of-words space cannot be learned. Deferral is evaluated at zero query cost; a per-query cost would move the operating point along the coverage-accuracy curve. Finally, the staged rejector is not jointly optimal - the classifier is not retrained to specialise on the region it keeps - which is the price paid for an interpretable, sklearn-only design.